RefundOps AI

Agentic Refund Operations | Product & Technical Design

Executive submission companion to the interactive RefundOps Studio demo

Version 1.0 • 01 November 2026 • Audience: AI Transformation Director / VP AI

Executive summary

RefundOps turns high-volume refund review into a governed decision workflow. Deterministic rules establish the financial safety boundary; a bounded investigation workflow organizes evidence and requests human judgment for exceptions. The offline Studio is a clickable experience, not a live LLM or payment system.

Business problem	Proposed outcome
Manual queue for straightforward requests	Safe low-risk recommendations and prioritized human review
Free-text policies and inconsistent decisions	Versioned policy rules, explicit assumptions and evidence links
Limited explainability	Per-case reason, applied rule and auditable reviewer actions
Risk from unsafe automation	Fail-closed controls, staged release, rollback and shadow evaluation

1 Product experience and user journeys

Roles and jobs-to-be-done

Role	Primary workflow
Refund Reviewer	Investigate a case, inspect evidence, answer a targeted question, decide or escalate
Refund Operations Manager	Draft rules, replay against history, inspect affected cases, publish with approval
Risk / Fraud Specialist	Investigate fraud flags and verify external evidence
Platform Administrator	Control permissions, integrations, observability and rollback

Core interaction

Overview → Case Workspace → Agent finding → Evidence → Execution trace → Human question → Safe action → Decision ledger. The primary action is contextual; avoid flooding the reviewer with equal-priority buttons.

Design principles

- Calm enterprise aesthetic: neutral canvas, navy typography, one restrained green accent; risk colors only when actionable.

- Evidence before action: each recommendation links to source fields, tool outputs and policy version.
- Progressive disclosure: show conclusion first; let reviewers expand trace, raw notes and related cases.
- Never imply autonomous execution when only a simulated trace is available.
- Accessibility: keyboard navigation, clear focus states, explicit status text and contrast.

2 System architecture

INPUT: pending.csv or same-schema API event → normalization and schema validation → versioned deterministic policy engine → auto-eligible recommendation OR human-review queue. A bounded investigation workflow reads case, policy and history evidence and produces a rationale and trace. Reviewers record actions; managers edit draft policies and replay history before publish.

Component	Current demo	Production target
UI	Self-contained HTML/JavaScript Studio; local browser state	React/TypeScript with server-side auth and accessible components
Decision engine	Python engine.py; deterministic thresholds and rules	Versioned Python service with signed policy bundles and typed schema
Workflow	Bounded read-only investigation steps; no live model calls	Durable state-machine / LangGraph-style orchestration with constrained tools
API / storage	Local Python API; CSV and local state	FastAPI, PostgreSQL, append-only audit events, object store for evidence
Model layer	Not connected	Optional server-side LLM for note extraction and explanation; JSON-schema outputs
External systems	None live	Order/OMS, warehouse, carrier, CRM, payments and fraud connectors

Agent boundaries

- Investigation agent: retrieves facts and extracts evidence; read-only by default.
- Risk specialist: combines deterministic risk flags and verified signals; cannot independently authorize money movement.
- Policy assistant: drafts a change and impact report; cannot publish it.
- Decision engine: final policy gate for any auto-approval recommendation. Payment execution is a separate privileged service with idempotency and approval controls.

3 Data and policy interpretation

The provided policy notes specify: 14-day change-of-mind window; photo for damage over $50; carrier evidence and investigation for never-arrived claims; fraud-team referral for fraud flags; full

refund for verified merchant picking errors; partial refunds require reviewer discretion. Chargeback cost is order amount plus $25.

Important distinction: $50 automatic cap, six-month tenure, maximum one prior refund and $300 manager threshold are conservative design assumptions, NOT explicit policy-note requirements. They must be confirmed with the business before production.

Known data-quality risks

- Historical human decisions are noisy labels, not ground truth; offline replay does not establish causal impact.
- Customer notes and support notes are untrusted text. Phrases such as “fraud team confirmed” or “warehouse confirmed” do not constitute independent verification.
- A “photo attached” text fragment is not verified image evidence. Missing evidence routes to human review.
- Fraud flags and malformed amounts must fail closed; use canonical status values and robust schema checks.

4 Guardrails and threat model

Threat / failure	Required control
Prompt injection in free-text notes	Treat notes as data, never instructions; no direct tool permission from note content
False fraud-team or warehouse confirmation	Require authenticated system-of-record confirmation, not keyword matching
LLM hallucinated facts	Cite tool evidence IDs, validate structured output, abstain on missing facts
Unapproved financial action	Policy gate + RBAC + human approval for exceptions + idempotent payment API
Policy drift / unsafe edits	Draft → replay → approval → staged publish; version pinning and instant rollback
Sensitive data exposure	Least privilege, encryption, retention policy, redact PII in prompts/logs
Agent loops / cost runaway	Tool allowlist, per-case token and tool-call budgets, timeouts, circuit breakers
Duplicate webhook / replay	Idempotency keys, event deduplication and transaction ledger

Critical pre-production fix

The current engine includes optional flags that can approve certain fraud or wrong-item cases based on matching phrases inside notes. These flags must remain disabled and must be replaced by authenticated external verification before any production deployment. No note-only evidence may unlock approval.

Action authorization matrix

Action	Agent	Reviewer	Manager / fraud
Read case and evidence	Allowed (scoped)	Allowed	Allowed

Recommend decision	Allowed	Allowed	Allowed
Approve low-risk recommendation	Never executes payment	Subject to policy	Subject to policy
Approve flagged fraud case	Blocked	Blocked until verified	Only via verified fraud workflow
Publish policy	Blocked	Blocked	Two-person approval in production
Execute refund payment	Blocked	No direct API access	Separate controlled service

5 Model strategy and technology choices

- Use deterministic Python logic for policy eligibility, limits, approvals and rejection guardrails; an LLM is not the source of truth.
- Use a small/efficient model for note classification, reason normalization and short grounded explanations, with a larger model only for genuinely ambiguous escalations.
- Enforce strict JSON schema, confidence/abstention criteria and evidence references; never trust model self-reported confidence alone.
- Use PostgreSQL for durable cases and policy versions; Redis/queue for asynchronous orchestration; OpenTelemetry traces and structured logs for observability.
- Build retrieval only if policy documents become numerous and complex; start with versioned policy objects rather than unnecessary vector infrastructure.

6 Cost model and unit economics

Illustrative planning assumptions, NOT vendor quotations or current API prices: 400 refund requests/week ≈ 1,733/month; 30% use an LLM extraction step; 1,200 input + 350 output tokens per invocation; assumed blended $1 per million input tokens and $4 per million output tokens. A 10% escalation subset uses a higher-cost pass assumed at $5/M input and $15/M output. These are scenario rates for budgeting only.

Monthly line item	Illustrative estimate
Primary extraction model	$1.35
Complex-case model	$3.99
LLM subtotal	$5.34
With 3× retries / prompt overhead	~$16.01
Hosting, DB, monitoring, support	Not priced; depends on deployment and SLA
Human reviewer labor	Must be measured using actual handling time and wage assumptions

Sensitivity: token costs scale approximately with request volume × AI invocation rate × tokens per invocation × unit price. At this workload, infrastructure, integrations, review labor and implementation are likely to dominate model-token cost. Recheck live vendor prices before procurement.

Business ROI framework

Net benefit/month = avoided manual review minutes × loaded labor cost/minute − model/infrastructure costs − added QA/review overhead − incremental fraud/chargeback losses. Do not treat auto-eligible volume as proven labor savings until an A/B or controlled pilot measures handling time and financial outcomes.

7 Evaluation and quality gates

Layer	Metric / gate
Decision safety	False-auto-approval rate on adjudicated holdout; zero critical fraud bypasses in prelaunch tests
Coverage	Auto-eligible percentage by reason, amount, customer cohort
Evidence quality	Supported claims / source citation accuracy; abstention rate
Human productivity	Median handling time, reviewer acceptance/override rate, escalation rate
Financial	Refund leakage, chargebacks per 1,000 cases, chargeback loss incl. $25 fee
Reliability	P95 latency, tool errors, duplicate-action rate, failed policy deploys
Fairness / drift	Segmented errors and approval rates; weekly policy and input-distribution drift

Validation plan: construct an adjudicated test set with independent fraud and policy labels, including adversarial notes, missing data, multilingual cases and contradictory evidence. Compare baseline rules vs candidate changes. Keep a frozen holdout and test every policy version. Historical approvals alone cannot prove correct decisions.

8 Rollout plan and rollback

Stage	Duration (illustrative)	Exit criteria
0. Offline baseline	1–2 weeks	Schema checks, rule tests, adjudicated sample, red-team prompt-injection tests
1. Shadow mode	2–3 weeks	No money movement; compare recommendations to reviewer outcomes; measure critical misses
2. Reviewer copilot	2–4 weeks	Human signs every decision; evidence trace, override reasons and adoption measured
3. Limited automation	2–4 weeks	Low-value low-risk cohort only; explicit risk approval; daily financial reconciliation
4. Controlled expansion	Ongoing	Thresholds increased only

	after sustained safety and business KPI evidence

Rollback triggers and mechanics

- Immediately disable auto-approval on any verified critical fraud bypass, payment mismatch or duplicate execution.
- Pause rollout when chargeback loss, override rate, latency or tool errors exceed pre-agreed thresholds; numerical thresholds require baseline agreement.
- Feature flag switches system to review-only; revert to last approved policy version; drain or quarantine in-flight actions.
- Preserve event logs and case IDs for reconciliation, root-cause analysis and regression tests; never erase an audit trail.

9 Operating model, governance and compliance

- Policy owner: Refund Operations; risk sign-off: Fraud/Finance; deployment owner: Engineering/SRE.
- Two-person review for high-impact policy changes, monthly risk review and incident response playbook.
- RBAC, SSO, retention and data-residency review before processing real customer PII.
- Audit records include actor, timestamp, input snapshot hash, policy/model version, tool evidence IDs, rationale and resulting action.
- Vendor security review and DPA for any external model; do not transmit card data or secrets in prompts.

10 Submission map and honest implementation status

Deliverable	Location / status
Interactive polished Studio	RefundOps_Studio_Live_Demo.html; standalone browser simulation
Runnable backend, CLI and tests	product/ within full submission package; deterministic engine
Executive deck	slides/RefundOps_AI_Executive_Pitch.pptx and .pdf
Design and architecture	This document + docs/ARCHITECTURE.md
Sample data	data/ CSV files
Actual 3–5 minute screen recording	NOT included; must record before submission
Public Git repository URL	NOT included; must publish before submission
Live LLM / real payment execution	NOT implemented; intentionally outside demo scope

Recommended demo script (3–5 min): open Studio → choose fraud-flagged case → inspect evidence and trace → show human escalation → open Policy Studio and replay → explain why automation cannot bypass the decision gate → finish with staged rollout and measurement.

References inside package

Source files: data/policy_notes.csv, data/history.csv, data/pending.csv, product/engine.py, product/api.py, docs/ARCHITECTURE.md and README.md. Cost rates and production architecture above are proposals/assumptions rather than facts supplied by the assignment.